

# P2406R0

## Fix ``counted_iterator`` interaction with input iterators

Published Proposal, 2021-07-11

**This version:**

[https://yehezekelshb.github.io/cpp\\_proposals/P2406-counted-iterator-and-input-iterators.html](https://yehezekelshb.github.io/cpp_proposals/P2406-counted-iterator-and-input-iterators.html)

**Authors:**

[Yehezkel Bernat](#)

[Yehuda Bernat](#)

**Audience:**

SG9 (RANGES)

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Source:**

[GitHub](#)

---

## Abstract

``counted_iterator`` increments its internal iterator even when reaching its own end, which makes it unusable in some cases, especially for input iterators. This paper suggest some changes to improve the situation

## Table of Contents

- 1 Revision History
- 2 Intro
- 3 Problem description
  - 3.1 Counter argument

### 3.2 The real problem

## 4 Design of suggested solution

### 4.1 The main changes

### 4.2 `random_access_iterator` case kept as-is

### 4.3 Open question: Constructing with 0 length

## 5 Proposed Wording

## 6 Effect on current state

## 7 Appendix

### References

Informative References

## § 1. Revision History

r0: initial revision

## § 2. Intro

Project Euler is a project with many mathematical-related questions that are intended to encourage the reader to write a small program to compute the result. In this case, one of the problems there, no. 37 [\[EULER\]](#), helped reveal a pitfall coming from the definition of `std::counted_iterator`.

## § 3. Problem description

Look at this example code [\[CE-FILTER\]](#):

```
#include <ranges>
#include <iostream>

namespace rv = std::views;

int main() {
    for (auto i : rv::iota(0)
```

```

        | rv::filter([](auto i) { return i < 10; })
        | rv::take(10))
    std::cout << i << '\n';
}

```

Compiler explorer gets a timeout when trying to run this simple example, instead of printing the numbers from 0 to 9. Running the same code locally, it runs for very long time. Tracking the roots of the issue, the problem is that `take` uses `counted_iterator` when the range isn't `random_access` and `counted_iterator` increments the internal iterator even if the counter has reached the requested count. In this case, the filter never returns when trying to increment it once again (at least not until `iota` reaches the UB case of signed overflow).

The example above is just for illustration, but we can think about cases where it isn't clear for the user how many items the filter is expected to return, so limiting the output count with `take` becomes dangerous and results in unexpected behavior.

The problem mentioned in the intro is one that actually describes a filter that return exactly 11 elements, so trying to use `take(11)` on it means the program never ends even while it got 11 elements already.

### § 3.1. Counter argument

If the user doesn't know how many items the filter returns, it's possible that the filter returns less items than what `take` tries to take, and never ends anyway. So one might claim that the user must know the number of items returned by the filter and then there is no point in using `take` when they are the same.

It still leaves the cases that there is a huge performance penalty on incrementing the iterator one additional time, but those are maybe considered less interesting.

### § 3.2. The real problem

The real problem we see is when using input ranges, e.g. `basic_istream_view`. In these cases, advancing the internal iterator means hanging forever if no additional input exists and the stream isn't closed or eating an additional input that can't be retrieved anymore. For example [\[CE-ISTREAM\]](#):

```

#include <ranges>
#include <iostream>
#include <sstream>

```

```

#include <cassert>

namespace rn = std::ranges;
namespace rv = rn::views;

int main()
{
    auto iss = std::istringstream("0 1 2");
    for (auto i : rn::istream_view<int>(iss)
         | rv::take(1))
        std::cout << i << '\n';
    auto i = 0;
    iss >> i;
    std::cout << i << std::endl; // flush it in case the assert fails
    assert(i == 1); // FAILS, i == 2
}

```

It means that one can't use ranges when parsing input, for example.

## § 4. Design of suggested solution

### § 4.1. The main changes

We suggest changing the behavior of `counted_iterator` operators around 0 length so it doesn't increment the internal iterator when reaching 0 length, and as a result doesn't decrement it when getting back from 0 to 1 length. This requires changing `base()` behavior too, including the return type to be by value.

### § 4.2. `random_access_iterator` case kept as-is

To reduce the amount of changes required, we keep the current behavior for `random_access_iterator` case, so we don't have to touch the additional operators defined only for this category. The rational behind it is that for `random_access_iterator` case we can expect the view to either have all the items ready or able to compute all of them efficiently, so it doesn't suffer from this issue.

### § 4.3. Open question: Constructing with 0 length

We don't have a good solution for the case that user constructs `counted_iterator` with `0` as argument for length. This puts it in an inconsistent internal state, as the next operations will be `base()` or `--`, and those expect the iterator to be one step back, with the changes suggested here.

Please note that `base()` and `--` are the only operations involving the state of the internal iterator and still legal for `counted_iterator` constructed with `n==0`;

Option 1: Require that if `n==0`, `i` must be decrementable, and actually decrement it in the c-tor. (This option assumes the only reason to create such an `counted_iteraor` is to decrement it anyway.)

Option 2: Require that if `n==0`, `i` must be "the one before" the actual iterator (leaving it to the user to decide how to handle, and if neither `--` nor `base()` are ever called on it, it doesn't matter what the user does).

Option 3: Mark this case internally (e.g. with `length=-1`) and handle specially when decrementing (`length` "jumps" to `1` after decrementing the internal iterator). Please note that `base()` doesn't need any special handling here.

## § 5. Proposed Wording

Note: Wording doesn't include any of the options suggested for the c-tor.

Under 23.5.6.3 [`counted.iter.access`]:

```
constexpr I base() const &;
```

Effects: Equivalent to: `return length ? current : next(current);`

Note: calling `base()` twice isn't safe when `I` isn't `forward_iterator`

```
constexpr const I& base() const &;
```

```
requires random_access_iterator<I>;
```

Effects: Equivalent to: `return current;`

```
constexpr I base() &&;
```

Returns: `std::move(length ? current : next(current)).`

```
constexpr I base() &&;
```

```
requires random_access_iterator<I>;
```

Returns: `std::move(current).`

Under 23.5.6.5 [**counted.iter.nav**]:

```
constexpr counted_iterator& operator++();
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
if (length > 1) ++current;
--length;
return *this;
```

```
constexpr counted_iterator& operator++();
```

```
requires random_access_iterator<I>;
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
++current;
--length;
return *this;
```

```
decltype(auto) operator++(int);
```

Preconditions: `length > 0`.

```
--length;
try { return current++; }
try { return length ? current++ : current; }
catch(...) { ++length; throw; }
```

```
constexpr counted_iterator operator++(int)
```

```
requires forward_iterator<I>;
```

Effects: Equivalent to:

```
counted_iterator tmp = *this;
++*this;
return tmp;
```

```
constexpr counted_iterator& operator--()
```

```
requires bidirectional_iterator<I>;
```

Effects: Equivalent to:

```
if (length) --current;
++length;
return *this;
```

```
constexpr counted_iterator& operator--()
```

```
requires random_access_iterator<I>;
```

Effects: Equivalent to:

```
--current;  
++length;  
return *this;
```

## § 6. Effect on current state

This breaks ABI (as every inline function does). If implementers don't accept this change, or if WG21 feels it's too late to apply it to C++20, we can add `lazy_counted_iterator` (or bikeshed a better name) which has the same behavior as `counted_iterator` except for the mentioned changes. In addition, it requires adding `lazy_take` and `lazy_counted_view` that uses it instead of `counted_iterator`.

In such a case, we suggest to consider requiring `forward_or_output_iterator` for `counted_iterator`, as it always does the wrong thing for `input_iterator` if it doesn't reach the `end` of the input (while for `forward_iterator` and `bidirectional_iterator` it's mainly just a possible performance issue and rarely an infinite loop, as mentioned, and for `output_iterator` the incrementing is usually no-op).

At the very least, we have to warn users against using `counted_iterator` and its consumers with `input_iterator`.

## § 7. Appendix

It's interesting to note that with optimizations enabled, gcc is able to "fix the issue" [\[CE-OPT\]](#). Our assumption is that it doesn't work with more complex cases, and anyway we can't count on optimization to fix such an issue. It's maybe even more interesting to see the mentioned optimization is not an optimizer bug, and when the filter will never return another number, it doesn't change the behavior [\[CE-OPT2\]](#).

## § References

### § Informative References

#### **[CE-FILTER]**

[filter+take problem example, Compiler Explorer](#). URL: <https://gcc.godbolt.org/z/cadsr1GMj>

#### **[CE-ISTREAM]**

[istream problem example, Compiler Explorer](#). URL: <https://gcc.godbolt.org/z/Eb8rdWYbP>

**[CE-OPT]**

Optimizer magic solves filter+take issue, Compiler Explorer. URL:  
<https://gcc.godbolt.org/z/4dahzG8Gz>

**[CE-OPT2]**

Optimizer is right when filter really never returns, Compiler Explorer. URL:  
<https://gcc.godbolt.org/z/PvMY8WeaT>

**[EULER]**

Truncatable primes, problem 37, Project Euler. URL: <https://projecteuler.net/problem=37>